

ECS 132 Fall 2020, Project

December 1, 2020

Your name: Gabriel Vazquez

Partner's name: Neema Nejad

Any other collaborator(s):

Instructions

1. Please refer to “ECS_132_Project.pdf” for details.
2. You may in no circumstances upload your project to private tutoring websites such as CourseHero or Chegg. Remember all material related to this course is a property of the University of California and posting them is a violation of the copyright laws.
3. If you refer to a source (either a book or the internet), you must cite it.
4. You may work in groups of size of at most 2. If you collaborate with others, you must list their names.
5. Write your answers in R Markdown and submit the knitted pdf on Gradescope; for due date and other details see the Homework Policy and Schedule.

3 Design

3.1

Alice and Bob decide to use the following modulation scheme to map the bits to the inter-packet delay. A delay of 0.25 is used to encode a bit 0 and delay of 0.75 is used to encode a bit 1. Write a short R code that will generate the modified packet stream that contains the secret message.

Answer

```
packetTimes = read.csv("Traffic_data_orig.csv")[2];
secretMsg = "this is a secret message";
binaryNumbers = numeric(length=(8*nchar(secretMsg)));

# convert character to binary numbers and store them in list.
for (i in 1:nchar(secretMsg)) {
  c = substr(secretMsg, i, i);
  # rawToBits returns the binary number with LSB first so need to reverse it.
  binary = rev(rawToBits(charToRaw(c)));
```

```

for (j in 1:length(binary)) {
  binaryNumbers[8*(i-1)+j] = if(binary[j] == 0) 0 else 1;
}
}

modifiedPacketStream = numeric(8*nchar(secretMsg)+1);
time = 0;
modifiedPacketStream[1] = time;

for (i in 1:(length(binaryNumbers))) {
  time = time + if(binaryNumbers[i] == 0) 0.25 else 0.75;
  modifiedPacketStream[i+1] = time;
}

print(modifiedPacketStream);

```

```

## [1] 0.00 0.25 1.00 1.75 2.50 2.75 3.50 3.75 4.00 4.25 5.00 5.75
## [13] 6.00 6.75 7.00 7.25 7.50 7.75 8.50 9.25 9.50 10.25 10.50 10.75
## [25] 11.50 11.75 12.50 13.25 14.00 14.25 14.50 15.25 16.00 16.25 16.50 17.25
## [37] 17.50 17.75 18.00 18.25 18.50 18.75 19.50 20.25 20.50 21.25 21.50 21.75
## [49] 22.50 22.75 23.50 24.25 25.00 25.25 25.50 26.25 27.00 27.25 27.50 28.25
## [61] 28.50 28.75 29.00 29.25 29.50 29.75 30.50 31.25 31.50 31.75 32.00 32.25
## [73] 33.00 33.25 33.50 34.25 34.50 34.75 35.00 35.25 35.50 35.75 36.50 37.25
## [85] 38.00 38.25 38.50 39.25 40.00 40.25 41.00 41.75 42.00 42.25 43.00 43.25
## [97] 44.00 44.25 45.00 45.75 46.00 46.25 46.50 47.25 48.00 48.25 49.00 49.75
## [109] 50.50 50.75 51.00 51.75 52.00 52.25 53.00 53.75 54.00 54.25 55.00 55.25
## [121] 56.00 56.25 57.00 57.75 58.50 58.75 59.50 59.75 60.00 60.25 60.50 61.25
## [133] 61.50 61.75 62.00 62.25 62.50 62.75 63.50 64.25 64.50 65.25 66.00 66.25
## [145] 67.00 67.25 68.00 68.75 69.00 69.25 70.00 70.25 71.00 71.25 72.00 72.75
## [157] 73.50 73.75 74.00 74.75 75.50 75.75 76.50 77.25 78.00 78.25 78.50 79.25
## [169] 80.00 80.25 81.00 81.75 82.00 82.25 82.50 82.75 83.50 83.75 84.50 85.25
## [181] 85.50 85.75 86.50 87.25 88.00 88.25 89.00 89.75 90.00 90.25 91.00 91.25
## [193] 92.00

```

3.2

Plot the histogram of the inter-packet delays of the overt packet stream. Plot the histogram of the covert packet stream. Will Eve be suspicious?

Answer

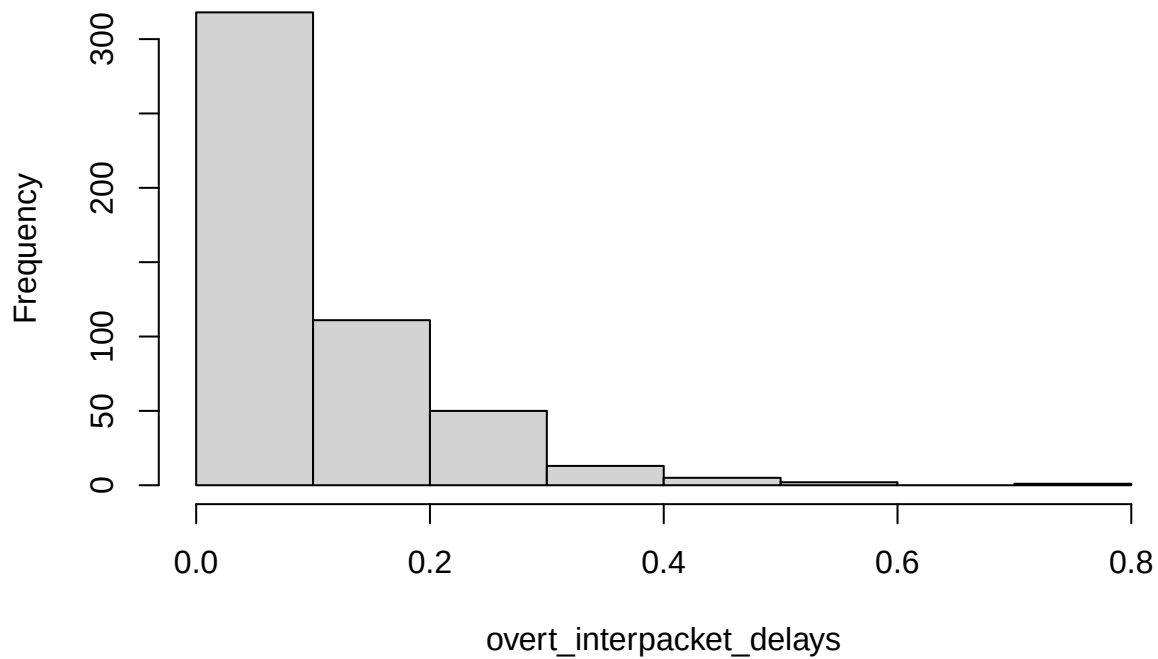
```

packetTimes = unlist(packetTimes, use.names = FALSE);

overt_interpacket_delays = packetTimes[2:length(packetTimes)] - packetTimes[1:(length(packetTimes)-1)]
covert_interpacket_delays = modifiedPacketStream[2:length(modifiedPacketStream)] - modifiedPacketStream[1:(length(modifiedPacketStream)-1)]
hist(overt_interpacket_delays, main="Overt Packet Stream Inter-Packet Delay Histogram");

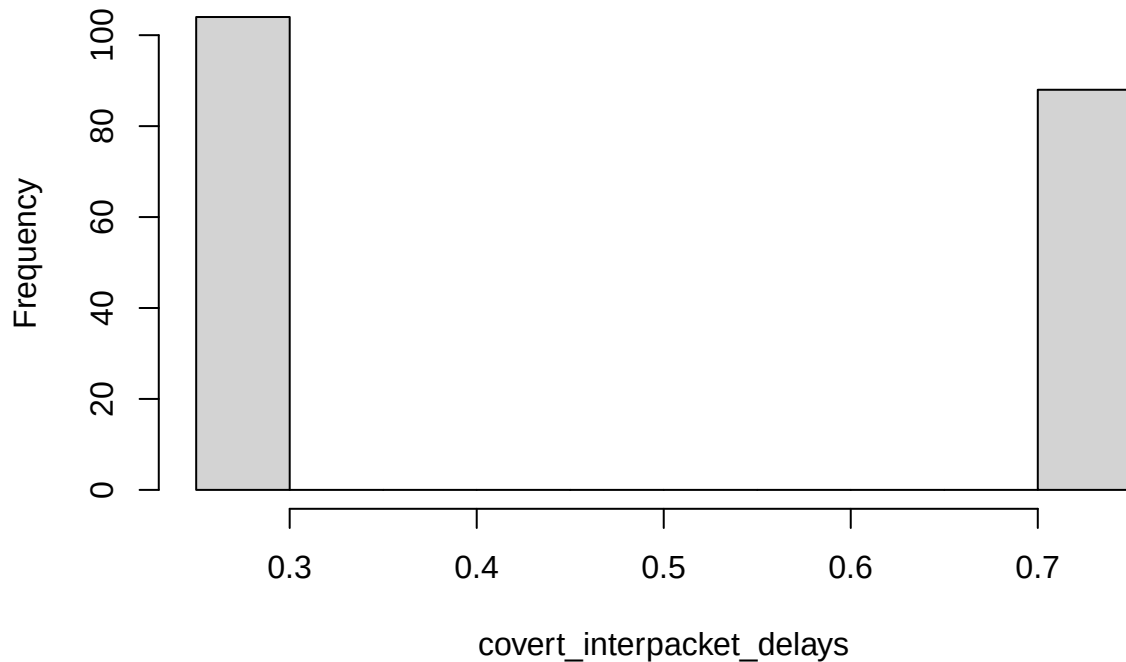
```

Overt Packet Stream Inter-Packet Delay Histogram



```
hist(covert_interpacket_delays, main="Covert Packet Stream Inter-Packet Delay Histogram")
```

Covert Packet Stream Inter-Packet Delay Histogram



Eve will definitely be suspicious, because the overt interpacket delays seems to follow an exponential distribution as opposed to the covert interpacket delays which takes on one of two values only. If Eve were to see this, Eve would definitely be suspicious.

3.3

Alice and Bob decide to use the following modulation scheme. Let m , $\min$, and $\max$ denote the median, min, and max of the inter-packet delay of the overt packet stream. If Alice needs to send a 0 she randomly generates a delay between $\min$ and m . If she wants to send a 1 she randomly generates a delay between m and $\max$. First, compute m , $\min$, and $\max$. Next, modify the code in Question 1, to generate the packet stream that contains the secret message.

Answer

```
# calculate median, min, and max of overt inner packet delays
m = mean(packetTimes[2:length(packetTimes)] - packetTimes[1:(length(packetTimes)-1)])
minimum = min(packetTimes[2:length(packetTimes)] - packetTimes[1:(length(packetTimes)-1)])
maximum = max(packetTimes[2:length(packetTimes)] - packetTimes[1:(length(packetTimes)-1)])

print("Median: ")

## [1] "Median: "

print(m)

## [1] 0.1011273

print("Minimum: ")

## [1] "Minimum: "

print(minimum)

## [1] 1e-05

print("Maximum: ")

## [1] "Maximum: "

print(maximum)

## [1] 0.73199

modifiedPacketStream2 = numeric(8*nchar(secretMsg)+1);
time = 0;
modifiedPacketStream2[1] = time;

for (i in 1:(length(binaryNumbers))) {
  time = time + if(binaryNumbers[i] == 0) runif(1, minimum, m) else runif(1, m, maximum);
  modifiedPacketStream2[i+1] = time;
}

print(modifiedPacketStream2);
```

```
## [1] 0.000000000 0.002978443 0.632996086 0.861751343 1.175456110
## [6] 1.216705199 1.560706097 1.633411755 1.697676804 1.709799931
## [11] 2.391681716 2.912351122 2.920356824 3.123182680 3.161017772
## [16] 3.224841799 3.316964721 3.412687596 3.705325793 4.027366934
## [21] 4.068922711 4.182334172 4.183621277 4.278811202 4.592221229
## [26] 4.638187455 5.147883998 5.772328158 6.097573051 6.181744931
## [31] 6.211686838 6.899752765 7.201922637 7.278776583 7.364889843
## [36] 7.966473723 8.056349844 8.095524491 8.128362023 8.214370948
## [41] 8.310325928 8.372676204 8.906562602 9.637752747 9.696559820
## [46] 10.128061863 10.191599141 10.250043690 10.362228667 10.447327296
## [51] 11.067069798 11.376259602 12.105928831 12.127470075 12.148591241
## [56] 12.786199221 13.298703534 13.319220603 13.399199777 13.584880653
## [61] 13.617025358 13.646834971 13.736298493 13.801010575 13.818554765
## [66] 13.882670656 14.372680313 14.630573713 14.645320050 14.706361855
## [71] 14.787247453 14.807615945 15.136884910 15.227380668 15.277970586
## [76] 15.997521554 16.098491093 16.102015365 16.185286699 16.204119472
## [81] 16.277541393 16.376587490 17.069142384 17.554686486 17.883955368
## [86] 17.969506117 17.997212852 18.577797943 18.875004523 18.950772171
## [91] 19.358181844 20.000129699 20.016673422 20.096094862 20.306583055
## [96] 20.404976342 20.725280386 20.792654822 21.013458653 21.117536155
## [101] 21.179874505 21.196043495 21.280019822 21.905422425 22.445042203
## [106] 22.473840496 23.043091063 23.507180390 24.186877562 24.283443053
## [111] 24.366490190 24.773827532 24.820561655 24.890080049 25.089194434
## [116] 25.468736982 25.512519451 25.557837889 25.762906115 25.780319336
## [121] 26.248664267 26.317996910 26.616576860 26.798551391 27.194633905
## [126] 27.268245457 27.491487705 27.583629070 27.585573904 27.608815672
## [131] 27.694996894 28.404265813 28.425135957 28.449684975 28.473664648
## [136] 28.488386952 28.509855536 28.599650588 29.044966178 29.421781938
## [141] 29.459372817 29.638925427 29.762639954 29.807100430 30.262343024
## [146] 30.293518885 30.634391732 30.787379487 30.794662740 30.888132835
## [151] 31.389542553 31.439152555 31.963316634 31.974502792 32.442150125
## [156] 33.164336027 33.429164180 33.466362284 33.515512600 33.882895935
## [161] 34.512910684 34.585018890 35.106968292 35.742183210 36.154709947
## [166] 36.212598847 36.249153879 36.568868176 36.838075260 36.856516437
## [171] 37.167743814 37.625661337 37.652225681 37.742713925 37.825536094
## [176] 37.900687130 38.111587219 38.136829954 38.336017244 38.511506767
## [181] 38.600918540 38.647045598 39.054860494 39.309999454 39.455681161
## [186] 39.500370233 39.648031467 39.937394280 39.988985819 40.003516554
## [191] 40.715697741 40.765603847 41.200519298
```

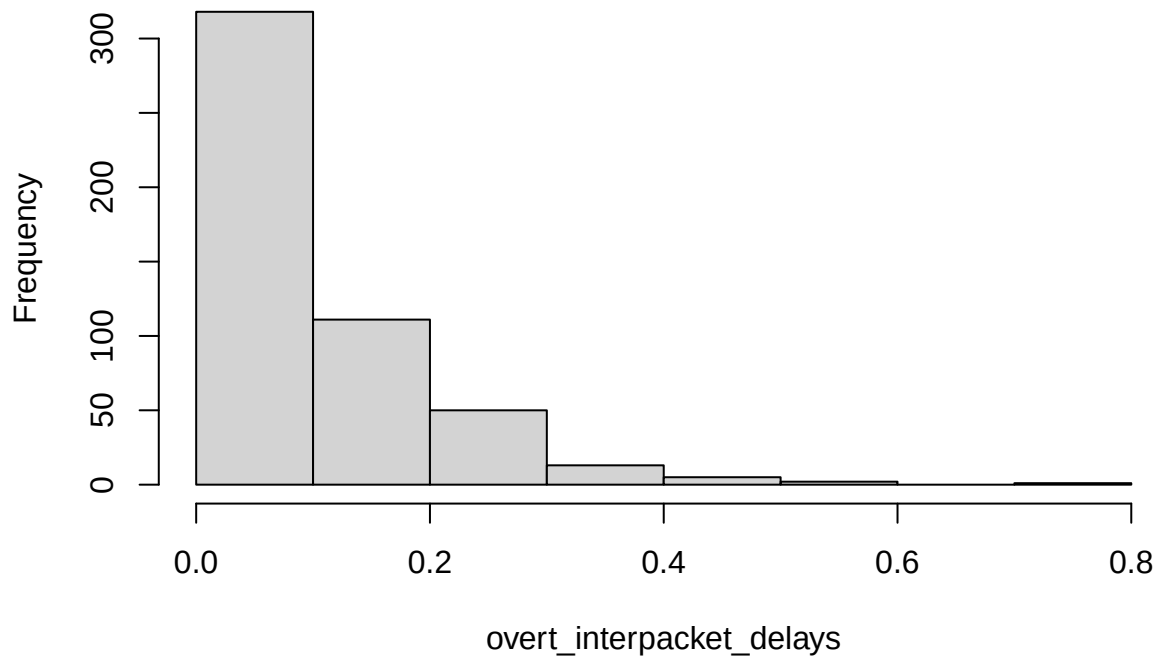
3.4

Plot the histogram of the inter-packet delays of the overt packet stream and that of the new covert packet stream. Do you think Eve will be suspicious?

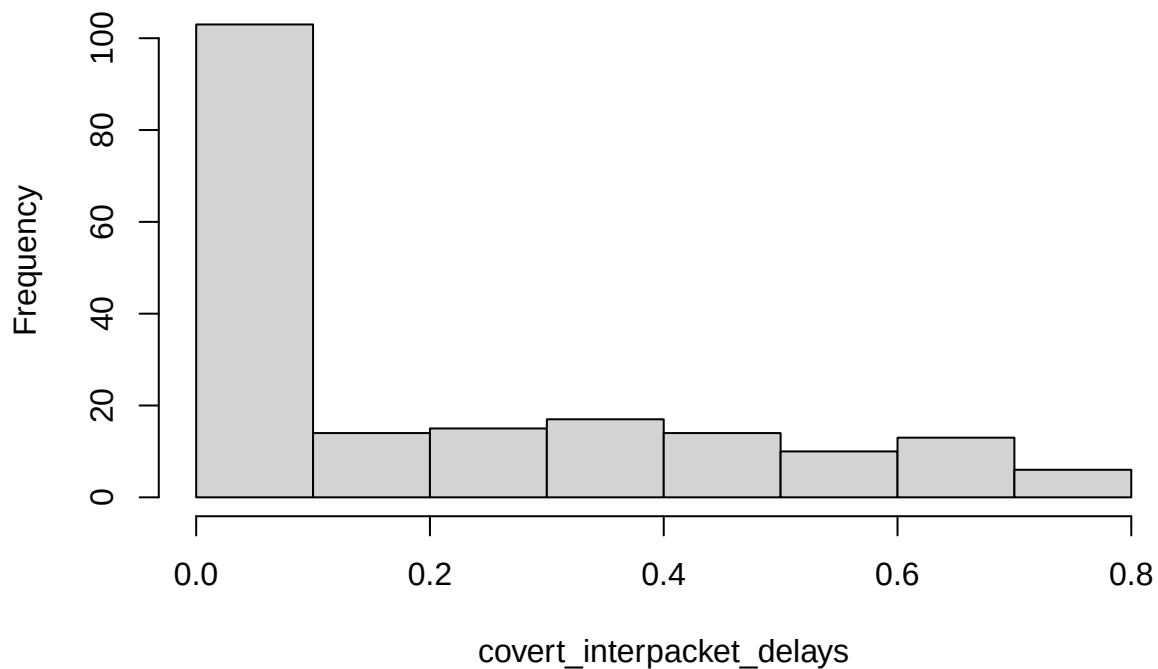
Answer

```
packetTimes = unlist(packetTimes, use.names = FALSE);

overt_interpacket_delays = packetTimes[2:length(packetTimes)] - packetTimes[1:(length(packetTimes)-1)]
covert_interpacket_delays = modifiedPacketStream2[2:length(modifiedPacketStream2)] - modifiedPacketStream2[1:(length(modifiedPacketStream2)-1)]
hist(overt_interpacket_delays, main="Overt Packet Stream Inter-Packet Delay Histogram");
```

Overt Packet Stream Inter-Packet Delay Histogram

```
hist(covert_interpacket_delays, main="Covert Packet Stream Inter-Packet Delay Histogram")
```

Covert Packet Stream Inter-Packet Delay Histogram

Eve will definitely be less suspicious because the frequency of the new covert inter packet delays are somewhat evenly distributed between various inter packet delay values while the previous histogram showed inter packet delays of only 0.25 and 0.75. Similar to the histogram of the overt inter packet delays, most of the covert inter packet delays are between 0 - 0.1. Since the histogram of the covert inter packet delays is much similar to the histogram of the overt inter packet delays, Eve will not be suspicious.

3.5

Answer the following questions briefly (in 1 or 2 sentences)

1. How can you improve upon the method in Question 3?

Answer: Instead of generating equally random delays between the min, median, and max, Alice could have generated random delays based on an exponential distribution that followed the overt inter packet distribution. From there, she could adjust/scale the min, median, and max values to determine how to send a 1 and 0 so the histogram of the covert inter packet delays almost replicates the histogram of the overt inter packet delays.

2. We assumed the Alice will buffer up the packets and we mentioned that it was unrealistic. Why?

Answer: As mentioned before, it's unrealistic for Alice to buffer up her own packets in a Skype call because the quality of the Skype call in terms of both audio and video would be severely compromised. A constant stream of packets must be sent without them being buffered to maintain good performance during the Skype call. Even then, Skype may be programmed to disconnect the call if no packets are being sent over a certain amount of time. Lastly, Alice may not have total control of the time between her inter packet delays.

3. We have assumed that the network does not alter the inter-packet delays. What would be the problem if it did? Can you suggest methods to mitigate the effect of the changes of the inter-packet delay (noise)?

Answer: If the network altered the inter-packet delays, then the hidden message could be obstructed by sending false 1's or 0's. This could be mitigated by checking how the network affects the covert inter packet delays then adjusting the values used to determine sending 1's and 0's so the noise is accounted for when calculating covert inter packet delays.

4 Implementation

4.1

For buffer size $B = 20$ we want to find out the probability of overflow and underflow, when the IPD follows the Exponential with $\lambda = 1$ pkts/sec and $i = 2, 6, 10, 14, 18$. Use message size $m = 16, 32$ bits. Tabulate the results. Remember that to determine the probability you need to run multiple (say 500) experiments for each parameter, i.e., for $B = 20, m = 16, i = 2$ run 500 experiments and determine the probability of overflow and underflow. Similarly for other values of i and m .

Answer

```
# return values: -1 = underflow, 0 = success, 1 = overflow
expSimulation <- function(maxBufferSize, currentBufferSize, messageSize) {
  median = log(2); # by default log -> ln
```

```

# create message.
bits = sample(0:1, messageSize, replace=T);

# create source packet times.
overt_packet_times = numeric(200);
overt_packet_times[1] = 0;
for (i in 2:200) {
  overt_packet_times[i] = overt_packet_times[i-1] + rexp(1);
}

time = 0;
for (bit in bits) {
  # figure out the delay needed for the current packet.
  delay = -1;
  while(T) {
    delay = rexp(1);

    if (bit == 0 && delay < median) {
      break;
    }

    if (bit == 1 && delay >= median) {
      break
    }
  }

  # send the packet
  currentBufferSize = currentBufferSize - 1;

  # sent a packet we don't have so underflow occurred.
  if (currentBufferSize < 0) {
    return(-1);
  }

  # buffer in arriving packets.
  currentBufferSize = currentBufferSize + length(overt_packet_times[(overt_packet_times >= time) & (overt_packet_times <= time + delay)]);

  # update time
  time = time + delay;

  # check for overflow.
  if (currentBufferSize > maxBufferSize) {
    return(1);
  }
}
return(0);
}

initBufferSizes = c(2,6,10,14,18);
m = c(16,32);

underflows16 = numeric(5);
underflows32 = numeric(5);

```

```

overflows16 = numeric(5);
underflows16 = numeric(5);

for (i in 1:length(initBufferSizes)) {
  overflows = 0;
  underflows = 0;
  for (j in 1:500) {
    value = expSimulation(20,initBufferSizes[i],16);
    if (value == 1) {
      overflows = overflows + 1;
    }
    else if (value == -1) {
      underflows = underflows + 1;
    }
  }
  underflows16[i] = underflows / 500;
  overflows16[i] = overflows / 500;
}

for (i in 1:length(initBufferSizes)) {
  overflows = 0;
  underflows = 0;
  for (j in 1:500) {
    value = expSimulation(20,initBufferSizes[i],32);
    if (value == 1) {
      overflows = overflows + 1;
    }
    else if (value == -1) {
      underflows = underflows + 1;
    }
  }
  underflows32[i] = underflows / 500;
  overflows32[i] = overflows / 500;
}

ans = matrix(c(underflows16,overflows16, underflows32,overflows32), nrow=4, ncol=5, byrow = T);
dimnames(ans) = list(c("underflow % (m=16)", "overflow % (m=16)", "underflow % (m=32)", "overflow % (m=32)"),
  c("i=2", "i=6", "i=10", "i=14", "i=18"))
print("Exponential Distribution")

## [1] "Exponential Distribution"

print(ans)

##           i=2   i=6  i=10  i=14  i=18
## underflow % (m=16) 0.560 0.160 0.008 0.000 0.000
## overflow % (m=16) 0.010 0.038 0.126 0.250 0.624
## underflow % (m=32) 0.684 0.362 0.122 0.028 0.002
## overflow % (m=32) 0.040 0.082 0.194 0.460 0.706

```

4.2

For buffer size $B = 20$ we want to find out the probability of overflow and underflow, when the IPD follows the Uniform distribution in the range $(0,1)$ and $i = 2, 6, 10, 14, 18$. Use message size $m = 16, 32$ bits. Tabulate

the results. Remember that to determine the probability you need to run multiple (say 500) experiments for each parameter, i.e., for $B = 20, m = 16, i = 2$ run 500 experiments and determine the probability of overflow and underflow. Similarly for other values of i and m .

Answer

```
# return values: -1 = underflow, 0 = success, 1 = overflow
unifSimulation <- function(maxBufferSize, currentBufferSize, messageSize) {
  median = 0.5;

  # create message.
  bits = sample(0:1, messageSize, replace=T);

  # create source packet times.
  overt_packet_times = numeric(200);
  overt_packet_times[1] = 0;
  for (i in 2:200) {
    overt_packet_times[i] = overt_packet_times[i-1] + runif(1);
  }

  time = 0;

  time = 0;
  for (bit in bits) {
    # figure out the delay needed for the current packet.
    delay = if(bit == 1) runif(1, 0.5,1) else runif(1,0,0.5);

    # send the packet
    currentBufferSize = currentBufferSize - 1;

    # check for underflow
    if (currentBufferSize < 0) {
      return(-1);
    }
    # buffer in arriving packets.
    currentBufferSize = currentBufferSize + length(overt_packet_times[(overt_packet_times >= time) & (overt_packet_times <= time + delay)])

    # update time
    time = time + delay;

    # check for overflow.
    if (currentBufferSize > maxBufferSize) {
      return(1);
    }
  }

  return(0);
}

# tabulate results and compute the probabilities
initBufferSizes = c(2,6,10,14,18);
m = c(16,32);
```

```

underflows16 = numeric(5);
underflows32 = numeric(5);
overflows16 = numeric(5);
overflows32 = numeric(5);

for (i in 1:length(initBufferSizes)) {
  overflows = 0;
  underflows = 0;
  for (j in 1:500) {
    value = unifSimulation(20,initBufferSizes[i],16);
    if (value == 1) {
      overflows = overflows + 1;
    }
    else if (value == -1) {
      underflows = underflows + 1;
    }
  }
  underflows16[i] = underflows / 500;
  overflows16[i] = overflows / 500;
}

for (i in 1:length(initBufferSizes)) {
  overflows = 0;
  underflows = 0;
  for (j in 1:500) {
    value = unifSimulation(20,initBufferSizes[i],32);
    if (value == 1) {
      overflows = overflows + 1;
    }
    else if (value == -1) {
      underflows = underflows + 1;
    }
  }
  underflows32[i] = underflows / 500;
  overflows32[i] = overflows / 500;
}

ans = matrix(c(underflows16,overflows16, underflows32,overflows32), nrow=4, ncol=5, byrow = T);
dimnames(ans) = list(c("underflow % (m=16)", "overflow % (m=16)", "underflow % (m=32)", "overflow % (m=32)"), c("i=2", "i=6", "i=10", "i=14", "i=18"));
print("Uniform Distribution")

## [1] "Uniform Distribution"

print(ans)

##           i=2   i=6  i=10  i=14  i=18
## underflow % (m=16) 0.430 0.022 0.000 0.000 0.000
## overflow % (m=16) 0.000 0.000 0.014 0.076 0.500
## underflow % (m=32) 0.542 0.130 0.006 0.002 0.000
## overflow % (m=32) 0.000 0.004 0.018 0.206 0.608

```

4.3

Propose methods to deal with buffer overflow and underflow.

Answer

Keeping the size of the message smaller generally keeps the percentage of overflow or underflow lower than messages with more bits. In both the Uniform and Exponential generation of inter-packet delays, the percentage of underflows and overflows are lower with each i . Additionally, $i = 10$ had the smallest chance of overflow and underflow for both Uniform and Exponential methods of generating inter-packet delays and for both message sizes. So keeping the message size smaller and the initial buffer size half the max buffer size ($i = 10 = 20 * 0.5$), then the percentage of underflows and overflows will drastically decrease.

To decrease the chance of overflows, the generated delays between packets could be scaled down so less packets are added to the buffer between a given time and a delay.

To decrease underflows we can let the sender be flexible with the delay they generated. For example, let's say we are sending packets using the uniform distribution with only one packet in the buffer, $t = 0$. If we send a 0 with delay = 0.25, and no packets arrive between $[0, 0.25]$ then we would normally consider it as underflow once we wait delay time, but the sender has a brief window of 0.25 seconds (0.5-delay). The sender could try and wait to see if packets arrive during this brief window, if any arrive we could send those and prevent underflow. We can't wait any longer than this period of time or else the 0 will be interpreted as a 1 to the receiver if delay becomes > 0.5 .

Lastly, the buffer size itself could be scaled bigger or smaller in accordance with the delay scaling mentioned previously to prevent underflows and overflows.